

SOLID PRINCIPLES

by

NeptuneLuke

Introduction

SOLID is an acronym for the object-oriented principles developed by Robert C. Martin. They are intended as guidelines for more readable, maintainable and extensible code. They can also be useful in identifying code smells and refactoring and adopting agile development practices.

S - Single responsibility principle

A class should have one and only one reason to change, meaning that a class should have only one job/responsibility.

O - Open-Closed principle

Objects or entities should be open for extension but closed for modification. Meaning that a class should be extendable without modifying the class itself.

L - Liskov substitution principle

Let $q(x)$ be a property provable about objects of x of type T . Then $q(y)$ should be provable for objects y of type S where S is a subtype of T . Meaning that every subclass or derived class should be substitutable for their parent class, without altering the software behavior.

I - Interface segregation principle

A client should never be forced to implement an interface that it doesn't use, or clients shouldn't be forced to depend on methods they do not use. Meaning that more specific interfaces are preferable over a single generic interface.

D - Dependency inversion principle

Entities must depend on abstractions, not on concretions. It states that the high-level module must not depend on the low-level module, but they should depend on abstractions. This principle allows for decoupling.